

# UK Grocery Store

## 6 Pre-Coding Documents

---

1. Product Requirements Document (PRD)
2. Technical Requirements Document (TRD)
  3. App Flow Document
  4. UI/UX Design Brief
5. Backend Schema Document
6. Implementation Plan

Project: UK STORE [ DEMO ]

Client: Trishulhub [ Office ]

Version: 2.0 - Updated with 8 Technical Resolutions

Generated: 2026-06-03

Framework: Supabase + Next.js + React Native Expo

# 1. Product Requirements Document (PRD)

## 1.1 App Overview

The UK Grocery Store is a hyperlocal grocery delivery platform designed to serve local communities across the United Kingdom. The application enables customers to browse a comprehensive catalog of grocery products, add items to their cart, and have their orders delivered to their doorstep within a defined delivery radius. The platform is specifically architected for a single-store deployment to accelerate the initial time-to-market, while the underlying database schema is designed with a mandatory **store\_id** relational key across all orders, products, and user profiles to ensure seamless multi-store scalability in the future without requiring database migrations or architectural overhauls.

The platform addresses a critical gap in the UK grocery market where independent grocery stores and small chains lack the digital infrastructure to compete with large supermarket delivery services like Tesco, Sainsbury's, and Ocado. By providing a turnkey digital storefront with integrated payment processing, real-time order tracking, and automated delivery management, the UK Grocery Store levels the playing field and enables local retailers to reach customers who increasingly prefer the convenience of online grocery shopping. Research from Statista indicates that Asda.com alone generated an estimated 5.42 billion USD in e-commerce net sales in 2025, underscoring the enormous market potential for grocery delivery platforms.

## 1.2 Problem Statement

Local and independent grocery stores in the UK face three fundamental challenges that prevent them from offering competitive online delivery services. First, the cost of building a custom e-commerce platform with integrated payment processing, real-time inventory management, and delivery logistics is prohibitively expensive for small businesses, often requiring enterprise-level budgets that are simply unavailable to independent retailers. Second, regulatory compliance in the UK grocery sector is complex and multi-faceted, encompassing HFSS (High Fat, Sugar, and Salt) advertising restrictions that take effect from January 2026, UK GDPR data protection requirements, and right-to-work verification for delivery couriers. Non-compliance with these regulations carries significant legal and reputational risks. Third, the technical complexity of implementing geospatial delivery zone management, real-time order tracking, and multi-role user management (customers, store owners, managers, pickers, and riders) demands a sophisticated technology stack that is typically beyond the reach of small grocery operations.

## 1.3 Target Users

| User Role | Description                               | Primary Need                                     |
|-----------|-------------------------------------------|--------------------------------------------------|
| Customer  | Local residents ordering groceries online | Easy browsing, fast checkout, real-time tracking |

|               |                                      |                                                              |
|---------------|--------------------------------------|--------------------------------------------------------------|
| Store Owner   | Business owner managing the store    | Full control over catalog, pricing, orders, and analytics    |
| Store Manager | Day-to-day operations manager        | Order management, staff coordination, inventory updates      |
| Picker        | In-store staff assembling orders     | Clear pick lists, item substitution handling, status updates |
| Rider         | Delivery courier transporting orders | Route navigation, delivery confirmation, earnings tracking   |

Table 1.1: User Roles and Primary Needs

## 1.4 Core Features with Priorities

Features are prioritized using the MoSCoW framework (Must, Should, Could, Won't) based on competitor analysis of Tesco, Sainsbury's, Ocado, and emerging quick-commerce apps, combined with the 8-week delivery timeline constraint.

### Must Have (MVP - Weeks 1-8)

- User registration and authentication via Supabase Auth (email, password, OTP)
- Product catalog browsing with category filtering and search
- Shopping cart with quantity management and item substitution preferences
- Stripe Checkout integration for secure payment processing
- Order placement with delivery slot selection
- Real-time order status tracking (Placed, Picking, Ready, Out for Delivery, Delivered)
- Admin dashboard for product, order, and delivery management
- HFSS compliance flagging (is\_hfss boolean) with automatic promotional exclusion
- Delivery pricing engine with configurable base fee, per-km charge, and free delivery threshold
- VAT calculation engine (0%, 5%, 20%) computed server-side

### Should Have (Post-MVP - Weeks 9-12)

- Rider GPS tracking with real-time location updates on customer app
- Push notifications for order status changes and promotional offers
- Order history with reorder functionality
- Product reviews and ratings system
- Multi-language support (English primary, with i18n infrastructure)
- Favourite items and shopping lists for quick reordering

## Could Have (Future Releases)

- AI-powered personalized product recommendations
- Subscription-based recurring orders (via Stripe Billing)
- Loyalty points and rewards program
- Recipe-based shopping (add all ingredients from a recipe)
- Social sharing of shopping lists
- Voice search for products

## Won't Have (V1)

- Multi-store frontend switching (schema-ready but UI is single-store)
- Third-party marketplace integration (Deliveroo, Just Eat)
- POS system integration with physical store inventory
- Augmented reality product preview
- Cryptocurrency payment options

## 1.5 User Stories

| As a...     | I want to...                         | So that...                                            |
|-------------|--------------------------------------|-------------------------------------------------------|
| Customer    | browse products by category          | I can quickly find the groceries I need               |
| Customer    | search for specific products         | I can find items without scrolling through categories |
| Customer    | see HFSS-compliant promotions        | I am not shown unhealthy items in promotional spots   |
| Customer    | add items to cart with substitutes   | I get alternatives if my first choice is unavailable  |
| Customer    | choose a delivery slot               | I know when my groceries will arrive                  |
| Customer    | pay with Stripe Checkout             | my payment is processed securely                      |
| Customer    | track my order in real-time          | I know when to expect my delivery                     |
| Store Owner | set delivery pricing thresholds      | I can offer free delivery on larger orders            |
| Store Owner | manage the product catalog           | I can keep inventory and prices up to date            |
| Picker      | see my assigned pick list            | I can assemble orders efficiently                     |
| Rider       | navigate to delivery addresses       | I can deliver orders on time                          |
| Rider       | upload ID documents for verification | I can comply with right-to-work regulations           |

Table 1.2: User Stories

### 1.6 Success Metrics (KPIs)

| Metric                      | Target        | Industry Benchmark            |
|-----------------------------|---------------|-------------------------------|
| Order Completion Rate       | > 95%         | 92-96% (Ocado, Tesco)         |
| Average Delivery Time       | < 60 minutes  | 45-90 min (quick commerce)    |
| Customer Retention (30-day) | > 40%         | 35-45% (grocery delivery)     |
| Cart Abandonment Rate       | < 60%         | 60-70% (grocery e-commerce)   |
| App Launch to First Order   | < 5 minutes   | 5-8 min (best-in-class)       |
| Order Accuracy Rate         | > 98%         | 95-98% (supermarket delivery) |
| Server Response Time        | < 200ms (p95) | < 300ms (industry standard)   |

Table 1.3: Key Performance Indicators

### 1.7 Competitive Advantage

Unlike large supermarket delivery platforms (Tesco, Sainsbury's, Ocado) that serve mass markets from centralized warehouses, the UK Grocery Store platform is purpose-built for independent and local grocery stores. The key differentiators include: first, a hyperlocal delivery model using PostGIS geospatial queries to define precise delivery zones around a single store, enabling faster delivery times and lower operational costs compared to warehouse-to-door models. Second, an admin-configurable delivery pricing engine that allows store owners to set base fees, per-km charges, and free delivery thresholds without code deployment, giving them real-time control over their delivery economics. Third, built-in HFSS compliance that automatically filters unhealthy products from promotional placements, providing legal compliance out-of-the-box from January 2026 when the UK's new advertising restrictions take effect. Fourth, a right-to-work verification system integrated directly into the rider onboarding flow, ensuring regulatory compliance for courier labor without additional software.

Research from quick-commerce industry analysis indicates that one-tap ordering interfaces, AI-powered personalized home screens, and hyper-fast search with smart filters are the top UI/UX trends for 2026. The platform is designed to incorporate these trends in its post-MVP roadmap while maintaining a clean, accessible interface for the MVP that prioritizes usability over novelty.

- [VERIFY: Store-specific POS integration requirements - confirm whether any existing POS system needs to be connected in V1]
- [VERIFY: Expected initial SKU count and catalog size - confirm 2,000+ SKUs is accurate for launch]

## 2. Technical Requirements Document (TRD)

### 2.1 Frontend Stack

The frontend is built on **Next.js 14+** using the App Router paradigm with TypeScript in strict mode. This framework provides server-side rendering for SEO-critical pages (product listings, category pages), client-side interactivity for the shopping cart and checkout flow, and API routes for backend communication. Tailwind CSS serves as the utility-first styling framework, enabling rapid UI development with a consistent design system. Zustand is the chosen state management library for client-side state (cart, user session, UI preferences) due to its minimal boilerplate and excellent TypeScript support.

For the mobile application, **React Native Expo** is selected over hybrid wrapper frameworks like CapacitorJS. This decision is driven by the grocery app's requirement for highly responsive catalog browsing, smooth gesture-based interactions, and resilient background data streaming for rider location updates. React Native Expo provides native UI rendering components, smoother gesture handling, and dedicated background processing threads, all on an open-source framework with zero licensing costs. The Expo managed workflow simplifies deployment to both iOS and Android app stores without native code complexity.

| Technology        | Version      | Purpose                                |
|-------------------|--------------|----------------------------------------|
| Next.js           | 14+          | Web application framework (App Router) |
| TypeScript        | 5.x (strict) | Type-safe development                  |
| Tailwind CSS      | 4.x          | Utility-first styling                  |
| Zustand           | 4.x          | Client-side state management           |
| React Native Expo | 50+          | Cross-platform mobile app              |
| Expo Router       | 3.x          | File-based mobile navigation           |

Table 2.1: Frontend Technology Stack

### 2.2 Backend Stack

The backend leverages **Supabase** as the unified backend-as-a-service platform, replacing the previously considered Prisma + Turso + NextAuth.js stack. Supabase provides a PostgreSQL relational database (up to 500 MB on the free tier), built-in authentication with email/password and OTP, real-time subscriptions for live order tracking, storage buckets for document uploads, and edge functions for serverless API endpoints. This consolidation dramatically reduces deployment complexity and eliminates the need to manage separate authentication, database, and storage services.

The built-in PostGIS extension is enabled via a single click in the Supabase dashboard, providing geospatial query capabilities for radius-based delivery zone calculations (e.g., finding all customers within a 5 km delivery radius). This eliminates the need for a secondary geospatial service and keeps all data within a single relational instance, simplifying data consistency and reducing infrastructure costs to zero during development and early production.

| Technology              | Purpose                  | Key Capability                                           |
|-------------------------|--------------------------|----------------------------------------------------------|
| Supabase PostgreSQL     | Core relational database | 500 MB free tier, PostGIS extension, Row Level Security  |
| Supabase Auth           | User authentication      | Email/password, OTP, social providers, JWT tokens        |
| Supabase Realtime       | Live order tracking      | WebSocket-based subscriptions for order status updates   |
| Supabase Storage        | Document uploads         | Rider ID/visa document storage with access control       |
| Supabase Edge Functions | Serverless API           | TypeScript Deno functions for payment webhooks, VAT calc |
| Next.js API Routes      | Application API layer    | RESTful endpoints for business logic orchestration       |

Table 2.2: Backend Technology Stack

## 2.3 Database

Supabase PostgreSQL serves as the sole database for the platform. The free tier accommodates up to 500 MB of pure text data, which is sufficient for a catalog of over 2,000 grocery SKUs and active order logs. PostgreSQL is the most popular database among developers with 55.6% adoption according to Stack Overflow surveys, and its native support for JSON, full-text search, and the PostGIS geospatial extension makes it ideal for the grocery delivery use case where location-based queries are core to the delivery zone management system.

Row Level Security (RLS) policies are enforced at the database level to ensure data isolation between user roles. Customers can only access their own orders and profiles. Store staff (managers, pickers) can access order data for their assigned store. Riders can only see orders assigned to them. The store owner has full access to all data within their store. This security model is enforced by PostgreSQL policies that are evaluated on every query, providing defense-in-depth beyond application-level checks.

## 2.4 Authentication

**Supabase Auth** replaces NextAuth.js v5 as the authentication provider, saving approximately 2 weeks of development time. Supabase Auth provides out-of-the-box support for email/password signup, OTP

(one-time password) verification, social OAuth providers (Google, Apple, Facebook), and JWT-based session management. The authentication flow is integrated directly with Row Level Security policies in PostgreSQL, meaning that every database query is automatically scoped to the authenticated user's role and permissions without additional application-level authorization code.

Session management uses Supabase's built-in JWT tokens with automatic refresh. The mobile app (React Native Expo) uses the Supabase React Native client with secure token storage on the device keychain. The web app uses the Supabase JavaScript client with HTTP-only cookies for CSRF protection. Password reset flows, email verification, and magic link authentication are all provided by Supabase Auth without custom implementation.

## 2.5 APIs and Integrations

| API / Service           | Purpose                                                | Pricing Model                                 |
|-------------------------|--------------------------------------------------------|-----------------------------------------------|
| Stripe Checkout         | Payment processing for grocery orders                  | Pay-as-you-go (1.5% + 20p per UK transaction) |
| Stripe Webhooks         | Payment event notifications (success, failure, refund) | Included with Stripe                          |
| Stripe Billing          | Future subscription/recurring order payments           | Pay-as-you-go (activatable in dashboard)      |
| PostGIS                 | Geospatial queries for delivery zone calculation       | Free (Supabase built-in extension)            |
| Supabase Realtime       | WebSocket subscriptions for live order tracking        | Free (included in Supabase tier)              |
| Expo Push Notifications | Mobile push notifications for order updates            | Free (Expo service)                           |
| Google Maps API         | Rider navigation and delivery address geocoding        | Free tier: 28,000 loads/month                 |
| SendGrid                | Transactional email (order confirmations, receipts)    | Free tier: 100 emails/day                     |

Table 2.3: API and Service Integrations

## 2.6 Security Requirements

- **UK GDPR Compliance:** All personal data (names, addresses, payment info) processed in accordance with UK GDPR. User consent required for data collection. Right to data deletion supported via Supabase Auth user deletion API.
- **PCI DSS:** Stripe Checkout handles all payment card data. The application never directly touches, stores, or transmits card numbers, ensuring PCI DSS compliance is inherited from Stripe's Level 1 certification.



- **Row Level Security:** Every database table enforces RLS policies scoped to user roles, preventing unauthorized data access even if the API layer is compromised.
- **HFSS Compliance:** The is\_hfss boolean flag on the products table is checked by frontend application logic to automatically exclude flagged items from promotional banners and top-tier discount spots, ensuring compliance with the UK HFSS advertising restrictions effective January 2026.
- **Right-to-Work Verification:** Rider document uploads (visas, IDs, licenses) are stored in private Supabase Storage buckets with access restricted to admin roles. Riders are blocked from the active order matching pool until their verification\_status transitions from "pending" to "approved" by a store administrator.
- **VAT Calculation:** All VAT calculations (0%, 5%, 20%) are performed server-side in Supabase Edge Functions to prevent client-side manipulation. The VAT rate is determined by the product's vat\_category field and cannot be altered by the client.

## 2.7 Performance Requirements

| Metric                    | Target            | Measurement                                    |
|---------------------------|-------------------|------------------------------------------------|
| Page Load Time (web)      | < 2 seconds       | Lighthouse Performance Score > 90              |
| API Response Time (p95)   | < 200ms           | Supabase dashboard monitoring                  |
| Mobile App Launch         | < 3 seconds       | Expo development builds profiling              |
| Real-time Update Latency  | < 500ms           | WebSocket message delivery time                |
| Image Load Time           | < 1 second        | CDN-backed product images via Supabase Storage |
| Database Query Time (p95) | < 50ms            | PostGIS queries with proper indexing           |
| Concurrent Users          | 500+ simultaneous | Load testing with k6 or Artillery              |

Table 2.4: Performance Targets

## 2.8 Cloud and Deployment

The web application is deployed on **Vercel**, which provides zero-configuration Next.js deployment, automatic preview deployments for pull requests, serverless API routes, and a global CDN for static assets. Vercel's free tier includes 100 GB bandwidth per month, which is sufficient for the initial launch phase. The React Native Expo mobile app is built using EAS (Expo Application Services) Build for generating production binaries, and EAS Submit for app store deployment to both Apple App Store and Google Play Store.

Supabase is the sole infrastructure provider for database, authentication, storage, and real-time capabilities. The free tier includes 500 MB database storage, 1 GB file storage, 50,000 monthly active

users for authentication, and 200 concurrent realtime connections. This is sufficient for the MVP launch with room for growth before requiring a paid plan. Environment variables for API keys and secrets are managed through Vercel's encrypted environment variable system and Supabase's project settings, with separate projects for development, staging, and production environments.

**[VERIFY: Confirm Vercel free tier bandwidth (100 GB/month) is sufficient for projected launch traffic]**

## 3. App Flow Document

### 3.1 All Screens / Pages

#### Customer App (Web + Mobile)

- Splash / Loading Screen
- Onboarding (3 slides: Browse, Order, Track)
- Login / Sign Up (Email, OTP, Social Auth)
- Forgot Password / Reset Password
- Email Verification Pending
- Home / Product Catalog (categories, featured, HFSS-filtered promos)
- Category Detail Page (product grid with filters)
- Product Detail Page (images, description, price, VAT, add to cart)
- Search Results Page (text search with autocomplete)
- Shopping Cart (items, quantities, substitutes, delivery slot picker)
- Checkout (address, delivery slot, payment via Stripe)
- Payment Success / Failure
- Order Confirmation (order ID, estimated delivery, receipt)
- Order Tracking (real-time status, rider location map)
- Order History (past orders with reorder button)
- Order Detail (items, status timeline, receipt)
- Profile / Account Settings
- Addresses Management (add/edit/delete delivery addresses)
- Favourites / Saved Items
- Notifications Center

#### Admin Dashboard (Web Only)

- Admin Login (Supabase Auth with role check)

- Dashboard Home (revenue, orders, delivery stats)
- Product Management (CRUD, categories, HFSS flag toggle, VAT category)
- Order Management (view all orders, assign pickers/riders, status updates)
- Delivery Management (delivery zones, pricing config, active riders)
- Rider Management (onboarding, document verification, status control)
- Customer Management (view customers, order history, support)
- Promotions Management (create/edit promos, HFSS auto-exclusion)
- Settings (store info, delivery pricing thresholds, VAT rates)
- Analytics (sales charts, popular products, delivery performance)

### **Picker App (Mobile)**

- Picker Login
- Assigned Orders List
- Pick List Detail (items to pick, location hints, substitute options)
- Item Picked Confirmation (scan barcode or manual confirm)
- Order Assembled Confirmation

### **Rider App (Mobile)**

- Rider Registration (with document upload for right-to-work)
- Rider Login (blocked until verification\_status = "approved")
- Available Deliveries (order list with delivery address and fee)
- Accept Delivery
- Navigation to Store (Google Maps integration)
- Pick Up Confirmation (scan order QR code)
- Navigation to Customer (Google Maps integration)
- Delivery Confirmation (photo proof, customer signature)
- Earnings Dashboard (completed deliveries, fees earned)

## **3.2 Primary User Journey: Customer Order Flow**

The customer order flow represents the most critical user journey in the application. It begins when a customer lands on the home screen and ends when the order is successfully delivered. Every step is designed to minimize friction and maximize conversion, drawing from industry best practices observed in leading UK grocery apps like Ocado and Tesco.

### **1. Home Screen**

Customer browses product categories and featured promotions. HFSS-flagged items are automatically excluded from promotional banners. Search bar provides instant autocomplete results. Quick-add

buttons allow adding frequently purchased items directly from the home screen.

## 2. Category / Product Browsing

Customer navigates to a category or uses search to find products. Products display name, price (including VAT), and availability status. Out-of-stock items are greyed out but visible to indicate catalog breadth.

## 3. Add to Cart

Customer taps "Add to Cart" on a product. A quantity selector allows adjusting amounts. Optional substitute preference can be set ("If unavailable, substitute with closest match" or "Do not substitute"). Cart badge updates in real-time.

## 4. Cart Review

Customer reviews all items, quantities, and substitute preferences. Estimated total is shown including VAT and estimated delivery fee based on the delivery address. "Proceed to Checkout" button is prominently displayed.

## 5. Checkout

Customer selects or adds a delivery address, chooses a delivery slot (if available), and reviews the order summary. Stripe Checkout is invoked for payment. The payment form is hosted by Stripe, ensuring PCI DSS compliance.

## 6. Payment Processing

Stripe processes the payment. On success, the order is created in the database with status "Placed" and the customer is redirected to the order confirmation screen. On failure, the customer is shown an error with the option to retry or use a different payment method.

## 7. Order Tracking

Customer sees real-time order status updates: Placed, Being Picked, Ready for Delivery, Out for Delivery (with rider location on map), Delivered. Push notifications are sent at each status transition.

## 8. Delivery Complete

Rider confirms delivery with photo proof. Customer receives a delivery confirmation notification with a summary and receipt. The customer is prompted to rate the delivery experience.

## 3.3 Empty States

Empty states are designed to guide new users toward their first meaningful action rather than displaying blank screens. Each empty state includes a friendly illustration, a short descriptive message, and a prominent call-to-action button. This approach is critical for user retention, as research shows that users who complete a first action within the first session are 3x more likely to return.

| Screen | Empty State Message | Call-to-Action |
|--------|---------------------|----------------|
|--------|---------------------|----------------|

|                  |                                             |                               |
|------------------|---------------------------------------------|-------------------------------|
| Cart             | Your cart is empty. Start adding groceries! | Browse Categories             |
| Order History    | No orders yet. Place your first order!      | Start Shopping                |
| Search Results   | No products found for your search.          | Try Different Keywords        |
| Favourites       | No saved items yet.                         | Add Favourites While Browsing |
| Rider Deliveries | No available deliveries right now.          | Refresh List                  |
| Picker Orders    | No orders to pick right now.                | Wait for Assignment           |

Table 3.1: Empty State Designs

### 3.4 Error States

Error states are categorized into three types: network errors (loss of connectivity), permission errors (unauthorized access), and validation errors (invalid form input). Each error state provides a clear explanation of what went wrong, why it happened, and what the user can do to resolve it. Critical actions (payment, order placement) implement retry logic with exponential backoff to handle transient network failures gracefully.

| Error Type           | Trigger                                 | User-Facing Message                                   | Recovery Action           |
|----------------------|-----------------------------------------|-------------------------------------------------------|---------------------------|
| Network Error        | No internet connection                  | Unable to connect. Check your internet and try again. | Retry Button              |
| Payment Failed       | Stripe returns card declined            | Payment was declined. Try a different card.           | Retry with New Card       |
| Session Expired      | JWT token refresh fails                 | Your session has expired. Please log in again.        | Redirect to Login         |
| Out of Stock         | Item becomes unavailable after cart add | An item in your cart is no longer available.          | Remove / Substitute       |
| Delivery Unavailable | No delivery slots for selected time     | No delivery slots available for this time.            | Choose Different Slot     |
| Rider Unverified     | Rider attempts login before approval    | Your account is pending verification.                 | Contact Support           |
| Rate Limited         | Too many API requests                   | Too many requests. Please wait a moment.              | Auto-retry after cooldown |

Table 3.2: Error State Handling

### 3.5 Login / Signup Flow

Authentication is handled by Supabase Auth with three signup methods: email and password, OTP (one-time password sent via email), and social OAuth (Google, Apple). New users are automatically assigned the "Customer" role. Store staff (managers, pickers, riders) are assigned their roles by the store owner through the admin dashboard after verification. The login flow includes a "Remember Me" option that extends the session duration from the default 1 hour to 7 days, reducing friction for returning users.

Password reset is handled by Supabase Auth's built-in flow: the user enters their email address, receives a password reset link, and sets a new password. Email verification is required for new accounts before they can place orders. The verification email is sent automatically by Supabase and includes a magic link that confirms the email address and redirects the user back to the app.

## 3.6 Payment Flow

The payment flow uses Stripe Checkout in server-redirected mode. When the customer clicks "Pay Now" on the checkout screen, the backend creates a Stripe Checkout Session with the order details (line items, VAT, delivery fee, total) and returns the session URL. The customer is redirected to Stripe's hosted payment page, which handles card collection, 3D Secure authentication (required for UK cards under SCA regulations), and payment processing. Upon completion, Stripe sends a webhook to the backend confirming the payment status, which triggers the order creation in the database and the confirmation notification to the customer.

Payment failure recovery is handled through three mechanisms. First, if the payment is declined by the bank, the customer is shown the decline reason (insufficient funds, expired card, etc.) and given the option to try a different payment method. Second, if the Stripe webhook fails to arrive (network issue), a scheduled cron job checks the status of pending payments every 5 minutes and updates the order accordingly. Third, if the customer abandons the checkout mid-payment, the cart is preserved with a "Resume Checkout" option visible on the cart screen for 24 hours, after which the pending Stripe session expires and the cart returns to normal state.

# 4. UI/UX Design Brief

## 4.1 Design Style and Mood

The design language follows a "**Fresh Local**" aesthetic that communicates trust, freshness, and community connection. The visual style draws inspiration from modern UK grocery apps like Ocado and Sainsbury's while establishing its own identity through warmer tones and a stronger emphasis on local community. The design prioritizes clarity and speed-of-use over decorative flourishes, reflecting the 2026 quick-commerce trend toward one-tap ordering interfaces and hyper-fast search with smart filters. The overall mood is approachable, efficient, and trustworthy, targeting a broad demographic from tech-savvy young professionals to older customers who may be ordering groceries online for the first time.

## 4.2 Color Palette

The color system uses a green primary to evoke freshness and trust (consistent with UK grocery industry norms), complemented by warm neutrals for readability and an orange accent for call-to-action elements. All color combinations meet WCAG 2.1 AA contrast requirements (minimum 4.5:1 for body text, 3:1 for large text and UI components).

| Role           | Color        | Hex     | Usage                                                |
|----------------|--------------|---------|------------------------------------------------------|
| Primary        | Fresh Green  | #16a34a | Primary buttons, links, active states, headers       |
| Primary Dark   | Forest Green | #15803d | Button hover, pressed states, emphasis               |
| Accent         | Warm Orange  | #f97316 | CTA buttons, badges, notifications, delivery markers |
| Background     | Off White    | #fafaf9 | Page backgrounds, card backgrounds                   |
| Surface        | Light Gray   | #f5f5f4 | Section backgrounds, alternating rows                |
| Text Primary   | Charcoal     | #1c1917 | Headings, body text, form labels                     |
| Text Secondary | Warm Gray    | #78716c | Descriptions, captions, muted text                   |
| Success        | Emerald      | #10b981 | Order confirmed, delivery complete                   |
| Warning        | Amber        | #f59e0b | Low stock, delivery delay warnings                   |
| Error          | Red          | #ef4444 | Payment failed, form errors, out of stock            |
| HFSS Badge     | Deep Orange  | #ea580c | HFSS indicator badge on product cards                |

Table 4.1: Color Palette

## 4.3 Typography

The typography system uses Inter as the primary sans-serif typeface for both web and mobile applications. Inter is selected for its excellent readability at small sizes, comprehensive weight range, and native support for tabular numbers (critical for price display alignment). For the mobile app, Inter is bundled with the Expo build. For the web, Inter is loaded from Google Fonts with font-display: swap for optimal loading performance.

| Element          | Font  | Size            | Weight         | Line Height |
|------------------|-------|-----------------|----------------|-------------|
| H1 Page Title    | Inter | 28px / 1.75rem  | Bold (700)     | 1.2         |
| H2 Section Title | Inter | 22px / 1.375rem | Semibold (600) | 1.3         |
| H3 Subsection    | Inter | 18px / 1.125rem | Semibold (600) | 1.4         |

|              |       |                  |                |     |
|--------------|-------|------------------|----------------|-----|
| Body Text    | Inter | 16px / 1rem      | Regular (400)  | 1.6 |
| Body Small   | Inter | 14px / 0.875rem  | Regular (400)  | 1.5 |
| Button Label | Inter | 16px / 1rem      | Semibold (600) | 1.0 |
| Price        | Inter | 18px / 1.125rem  | Bold (700)     | 1.2 |
| Price Small  | Inter | 14px / 0.875rem  | Semibold (600) | 1.2 |
| Caption      | Inter | 12px / 0.75rem   | Regular (400)  | 1.4 |
| Badge        | Inter | 11px / 0.6875rem | Semibold (600) | 1.0 |

Table 4.2: Typography Scale

## 4.4 Component Style Guide

### Buttons

- **Primary Button:** Background: Fresh Green (#16a34a), Text: White, Border-radius: 8px, Padding: 12px 24px, Font: Inter Semibold 16px. Hover: darken 10%. Active: darken 15%. Disabled: 50% opacity.
- **Secondary Button:** Background: Transparent, Text: Fresh Green, Border: 1.5px Fresh Green, Border-radius: 8px, Padding: 11px 23px.
- **Accent Button (CTA):** Background: Warm Orange (#f97316), Text: White, Border-radius: 8px, Padding: 12px 24px. Used exclusively for "Add to Cart", "Place Order", and "Pay Now".
- **Icon Button:** 40x40px circle, transparent background, icon in Text Secondary color. Hover: Surface background.

### Cards

- **Product Card:** Background: White, Border-radius: 12px, Box-shadow: 0 1px 3px rgba(0,0,0,0.08). Image: 200x200px, aspect-ratio: 1. Product name (2 lines max), price with VAT, "Add" button. HFSS badge in top-right corner if is\_hfss = true.
- **Order Card:** Background: White, Border-radius: 12px, Left border: 4px status color (green = delivered, blue = in progress, orange = pending). Order ID, date, item count, total, status badge.
- **Category Card:** Background: Surface (#f5f5f4), Border-radius: 12px, Category icon (48x48px), Category name, Item count. Hover: subtle scale (1.02).

### Form Inputs

- **Text Input:** Background: White, Border: 1px #d6d3d1, Border-radius: 8px, Padding: 12px 16px, Font: Inter Regular 16px. Focus: Border color changes to Primary, with 2px ring in Primary at 20% opacity. Error: Border color changes to Error, error message below in Error color.
- **Dropdown:** Same as text input with chevron-down icon. Options panel: white background, box-shadow, max-height: 240px with scroll.



- **Quantity Selector:** 3-part control: [-] [number] [+]. Buttons: 32x32px, rounded, border: 1px #d6d3d1. Number: Inter Bold 16px, min-width 32px, center-aligned.

## 4.5 Layout Rules and Grid System

The layout system uses a 12-column grid on desktop (breakpoint  $\geq 1024\text{px}$ ), 4-column grid on tablet (768px - 1023px), and a single-column stack on mobile ( $< 768\text{px}$ ). The maximum content width is 1280px, centered with auto margins. Spacing follows a 4px base unit system: 4, 8, 12, 16, 20, 24, 32, 40, 48, 64px. All component spacing, padding, and margins are multiples of 4px to maintain visual consistency.

| Breakpoint                       | Columns | Gutter | Margin | Layout                            |
|----------------------------------|---------|--------|--------|-----------------------------------|
| Mobile ( $< 768\text{px}$ )      | 1       | 0      | 16px   | Single column stack               |
| Tablet (768-1023px)              | 4       | 16px   | 24px   | 2-column product grid             |
| Desktop ( $\geq 1024\text{px}$ ) | 12      | 24px   | 32px   | 3-4 column product grid + sidebar |

Table 4.3: Responsive Breakpoints

## 4.6 Mobile-Specific Behaviors

The React Native Expo mobile app implements platform-specific adaptations beyond responsive layout. iOS devices use native SF Symbols for tab bar icons and the iOS-style large title navigation bar. Android devices use Material Design 3 components including the Material bottom navigation bar and ripple touch feedback. Both platforms share the same visual language defined in this brief but respect platform conventions for navigation, gestures, and system UI integration.

- **Bottom Navigation:** 5 tabs: Home, Categories, Search, Cart (with badge), Profile. Cart badge shows real-time item count.
- **Swipe Gestures:** Swipe left on cart item to remove. Swipe right on order to reorder. Pull-to-refresh on all list screens.
- **Haptic Feedback:** Light impact on add-to-cart, medium impact on order placed, success notification on delivery confirmed.
- **Offline Support:** Product catalog cached locally for 24 hours. Cart persisted in local storage. Order status polled every 30 seconds when WebSocket is disconnected.

## 4.7 Design Inspiration

- **Ocado:** Clean product grid, prominent delivery slot selector, real-time order tracking with map. The benchmark for UK grocery app UX.

- **Sainsbury's:** Category navigation with visual icons, smart substitutions UI, Nectar points integration.
  - **Gorillas / Getir (Quick Commerce):** One-tap reorder, minimal checkout steps, bold product photography, countdown timers.
  - **Deliveroo:** Rider tracking map, estimated arrival countdown, clean status timeline.
- [VERIFY: Confirm whether the brand has an existing logo, color scheme, or style guide that must be followed]

## 5. Backend Schema Document

### 5.1 Overview

The database schema is designed for Supabase PostgreSQL with the PostGIS extension enabled. All tables use UUID primary keys for global uniqueness (critical for future multi-store synchronization), include created\_at and updated\_at timestamps, and enforce Row Level Security policies scoped to user roles. The schema includes a mandatory store\_id foreign key on all business tables to support future multi-store expansion without schema migration. Foreign keys use ON DELETE CASCADE for dependent records and ON DELETE RESTRICT for protected references.

### 5.2 Core Tables

Table: stores

| Column    | Type                   | Constraints                    | Description                   |
|-----------|------------------------|--------------------------------|-------------------------------|
| id        | UUID                   | PK, DEFAULT uuid_generate_v4() | Unique store identifier       |
| name      | VARCHAR(255)           | NOT NULL                       | Store display name            |
| slug      | VARCHAR(100)           | UNIQUE, NOT NULL               | URL-friendly store identifier |
| address   | TEXT                   | NOT NULL                       | Full store address            |
| latitude  | DECIMAL(10,8)          | NOT NULL                       | Store latitude for PostGIS    |
| longitude | DECIMAL(11,8)          | NOT NULL                       | Store longitude for PostGIS   |
| location  | GEOGRAPHY(POINT, 4326) | GENERATED                      | PostGIS geography point       |
| phone     | VARCHAR(20)            |                                | Store contact phone           |

|                         |               |               |                               |
|-------------------------|---------------|---------------|-------------------------------|
| email                   | VARCHAR(255)  |               | Store contact email           |
| base_delivery_fee       | DECIMAL(10,2) | DEFAULT 3.50  | Base delivery charge in GBP   |
| per_km_charge           | DECIMAL(10,2) | DEFAULT 0.50  | Per-km delivery surcharge     |
| free_delivery_threshold | DECIMAL(10,2) | DEFAULT 20.00 | Order value for free delivery |
| delivery_radius_km      | DECIMAL(5,2)  | DEFAULT 5.00  | Max delivery distance         |
| is_active               | BOOLEAN       | DEFAULT true  | Store operational status      |
| created_at              | TIMESTAMPTZ   | DEFAULT now() | Record creation timestamp     |
| updated_at              | TIMESTAMPTZ   | DEFAULT now() | Record update timestamp       |

Table 5.1: stores

**Table: profiles**

| Column     | Type         | Constraints                                                  | Description                          |
|------------|--------------|--------------------------------------------------------------|--------------------------------------|
| id         | UUID         | PK, REFERENCES auth.users(id) ON DELETE CASCADE              | User ID (linked to Supabase Auth)    |
| store_id   | UUID         | FK stores(id), NOT NULL                                      | Assigned store (multi-store ready)   |
| email      | VARCHAR(255) | NOT NULL                                                     | User email address                   |
| full_name  | VARCHAR(255) | NOT NULL                                                     | Display name                         |
| phone      | VARCHAR(20)  |                                                              | Contact phone number                 |
| role       | VARCHAR(20)  | NOT NULL, CHECK IN (customer, owner, manager, picker, rider) | User role                            |
| avatar_url | TEXT         |                                                              | Profile image URL (Supabase Storage) |
| is_active  | BOOLEAN      | DEFAULT true                                                 | Account active status                |
| created_at | TIMESTAMPTZ  | DEFAULT now()                                                | Record creation timestamp            |
| updated_at | TIMESTAMPTZ  | DEFAULT now()                                                | Record update timestamp              |

Table 5.2: profiles

**Table: categories**

| Column      | Type         | Constraints                             | Description               |
|-------------|--------------|-----------------------------------------|---------------------------|
| id          | UUID         | PK, DEFAULT<br>uuid_generate_v4()       | Category unique ID        |
| store_id    | UUID         | FK stores(id), NOT NULL                 | Owning store              |
| name        | VARCHAR(100) | NOT NULL                                | Category display name     |
| slug        | VARCHAR(100) | NOT NULL                                | URL-friendly identifier   |
| description | TEXT         |                                         | Category description      |
| image_url   | TEXT         |                                         | Category image URL        |
| parent_id   | UUID         | FK categories(id) ON<br>DELETE SET NULL | Parent category (nested)  |
| sort_order  | INTEGER      | DEFAULT 0                               | Display order             |
| is_active   | BOOLEAN      | DEFAULT true                            | Category visibility       |
| created_at  | TIMESTAMPTZ  | DEFAULT now()                           | Record creation timestamp |

Table 5.3: categories

### Table: products

| Column      | Type          | Constraints                                    | Description                  |
|-------------|---------------|------------------------------------------------|------------------------------|
| id          | UUID          | PK, DEFAULT<br>uuid_generate_v4()              | Product unique ID            |
| store_id    | UUID          | FK stores(id), NOT NULL                        | Owning store                 |
| category_id | UUID          | FK categories(id), NOT NULL                    | Product category             |
| name        | VARCHAR(255)  | NOT NULL                                       | Product display name         |
| slug        | VARCHAR(255)  | NOT NULL                                       | URL-friendly identifier      |
| description | TEXT          |                                                | Product description          |
| price       | DECIMAL(10,2) | NOT NULL                                       | Price including VAT (GBP)    |
| vat_rate    | DECIMAL(5,4)  | NOT NULL, CHECK IN<br>(0.0000, 0.0500, 0.2000) | VAT rate (0%, 5%, 20%)       |
| is_hfss     | BOOLEAN       | DEFAULT false                                  | HFSS compliance flag         |
| image_url   | TEXT          |                                                | Product image URL            |
| barcode     | VARCHAR(50)   |                                                | Product barcode (EAN/GTIN)   |
| unit        | VARCHAR(20)   | DEFAULT "each"                                 | Unit (each, kg, litre, pack) |

|                |              |               |                               |
|----------------|--------------|---------------|-------------------------------|
| weight_kg      | DECIMAL(8,3) |               | Product weight in kg          |
| is_available   | BOOLEAN      | DEFAULT true  | Availability status           |
| stock_quantity | INTEGER      | DEFAULT 0     | Current stock count           |
| is_featured    | BOOLEAN      | DEFAULT false | Featured on homepage          |
| sort_order     | INTEGER      | DEFAULT 0     | Display order within category |
| created_at     | TIMESTAMPTZ  | DEFAULT now() | Record creation timestamp     |
| updated_at     | TIMESTAMPTZ  | DEFAULT now() | Record update timestamp       |

Table 5.4: products

### Table: addresses

| Column         | Type                   | Constraints                                       | Description                      |
|----------------|------------------------|---------------------------------------------------|----------------------------------|
| id             | UUID                   | PK, DEFAULT<br>uuid_generate_v4()                 | Address unique ID                |
| user_id        | UUID                   | FK profiles(id) ON<br>DELETE CASCADE, NOT<br>NULL | Owning user                      |
| label          | VARCHAR(50)            |                                                   | Address label (Home, Work, etc.) |
| address_line_1 | VARCHAR(255)           | NOT NULL                                          | First line of address            |
| address_line_2 | VARCHAR(255)           |                                                   | Second line of address           |
| city           | VARCHAR(100)           | NOT NULL                                          | City                             |
| postcode       | VARCHAR(10)            | NOT NULL                                          | UK postcode                      |
| latitude       | DECIMAL(10,8)          |                                                   | Address latitude                 |
| longitude      | DECIMAL(11,8)          |                                                   | Address longitude                |
| location       | GEOGRAPHY(POINT, 4326) | GENERATED                                         | PostGIS geography point          |
| is_default     | BOOLEAN                | DEFAULT false                                     | Default delivery address         |
| created_at     | TIMESTAMPTZ            | DEFAULT now()                                     | Record creation timestamp        |

Table 5.5: addresses

### Table: orders

| Column | Type | Constraints | Description |
|--------|------|-------------|-------------|
|--------|------|-------------|-------------|

|                          |               |                                                                                              |                                            |
|--------------------------|---------------|----------------------------------------------------------------------------------------------|--------------------------------------------|
| id                       | UUID          | PK, DEFAULT<br>uuid_generate_v4()                                                            | Order unique ID                            |
| store_id                 | UUID          | FK stores(id), NOT NULL                                                                      | Fulfilling store                           |
| customer_id              | UUID          | FK profiles(id), NOT NULL                                                                    | Ordering customer                          |
| picker_id                | UUID          | FK profiles(id)                                                                              | Assigned picker                            |
| rider_id                 | UUID          | FK profiles(id)                                                                              | Assigned rider                             |
| address_id               | UUID          | FK addresses(id), NOT NULL                                                                   | Delivery address                           |
| status                   | VARCHAR(20)   | NOT NULL, CHECK IN<br>(placed, picking, ready,<br>out_for_delivery, delivered,<br>cancelled) | Order status                               |
| subtotal                 | DECIMAL(10,2) | NOT NULL                                                                                     | Subtotal before VAT and<br>delivery        |
| vat_amount               | DECIMAL(10,2) | NOT NULL                                                                                     | Total VAT amount                           |
| delivery_fee             | DECIMAL(10,2) | NOT NULL                                                                                     | Delivery fee charged                       |
| total                    | DECIMAL(10,2) | NOT NULL                                                                                     | Grand total (subtotal + VAT +<br>delivery) |
| stripe_session_id        | VARCHAR(255)  |                                                                                              | Stripe Checkout session ID                 |
| stripe_payment_intent_id | VARCHAR(255)  |                                                                                              | Stripe payment intent ID                   |
| payment_status           | VARCHAR(20)   | DEFAULT pending, CHECK<br>IN (pending, paid, failed,<br>refunded)                            | Payment status                             |
| delivery_slot            | TIMESTAMPTZ   |                                                                                              | Requested delivery time slot               |
| notes                    | TEXT          |                                                                                              | Customer delivery instructions             |
| created_at               | TIMESTAMPTZ   | DEFAULT now()                                                                                | Order creation timestamp                   |
| updated_at               | TIMESTAMPTZ   | DEFAULT now()                                                                                | Order update timestamp                     |

Table 5.6: orders

**Table: order\_items**

| Column | Type | Constraints                       | Description         |
|--------|------|-----------------------------------|---------------------|
| id     | UUID | PK, DEFAULT<br>uuid_generate_v4() | Line item unique ID |

|                       |               |                                             |                                  |
|-----------------------|---------------|---------------------------------------------|----------------------------------|
| order_id              | UUID          | FK orders(id) ON DELETE CASCADE, NOT NULL   | Parent order                     |
| product_id            | UUID          | FK products(id), NOT NULL                   | Product reference                |
| product_name          | VARCHAR(255)  | NOT NULL                                    | Product name at time of order    |
| quantity              | INTEGER       | NOT NULL, CHECK > 0                         | Quantity ordered                 |
| unit_price            | DECIMAL(10,2) | NOT NULL                                    | Price per unit at time of order  |
| vat_rate              | DECIMAL(5,4)  | NOT NULL                                    | VAT rate applied                 |
| vat_amount            | DECIMAL(10,2) | NOT NULL                                    | VAT amount for this line         |
| subtotal              | DECIMAL(10,2) | NOT NULL                                    | Line subtotal (qty x unit_price) |
| substitute_preference | VARCHAR(20)   | CHECK IN (closest_match, do_not_substitute) | Substitution preference          |
| substituted_with      | UUID          | FK products(id)                             | Actual product substituted       |
| picked                | BOOLEAN       | DEFAULT false                               | Whether item was picked          |

Table 5.7: order\_items

**Table: rider\_verifications**

| Column              | Type        | Constraints                                                          | Description                                |
|---------------------|-------------|----------------------------------------------------------------------|--------------------------------------------|
| id                  | UUID        | PK, DEFAULT<br>uuid_generate_v4()                                    | Verification record ID                     |
| rider_id            | UUID        | FK profiles(id) ON DELETE CASCADE, NOT NULL                          | Rider being verified                       |
| document_type       | VARCHAR(30) | NOT NULL, CHECK IN<br>(passport, driving_license, visa, national_id) | Type of document                           |
| document_url        | TEXT        | NOT NULL                                                             | Supabase Storage URL for uploaded document |
| verification_status | VARCHAR(20) | DEFAULT pending, CHECK IN (pending, approved, rejected)              | Verification status                        |
| verified_by         | UUID        | FK profiles(id)                                                      | Admin who verified                         |
| verified_at         | TIMESTAMPTZ |                                                                      | Timestamp of verification decision         |
| rejection_reason    | TEXT        |                                                                      | Reason if rejected                         |

|            |             |               |                  |
|------------|-------------|---------------|------------------|
| created_at | TIMESTAMPTZ | DEFAULT now() | Upload timestamp |
|------------|-------------|---------------|------------------|

Table 5.8: rider\_verifications

## 5.3 Indexes

Indexes are designed based on expected query patterns. The primary performance-critical queries involve geospatial lookups for delivery zone validation, product searches within categories, and order status updates. The PostGIS GiST index on the location columns enables radius-based queries such as "find all stores within 5 km of this postcode".

| Table       | Index                       | Type                                  | Purpose                       |
|-------------|-----------------------------|---------------------------------------|-------------------------------|
| stores      | idx_stores_location         | GiST (location)                       | Radius-based store lookup     |
| products    | idx_products_store_category | B-tree (store_id, category_id)        | Category browsing per store   |
| products    | idx_products_name_trgm      | GIN (name gin_trgm_ops)               | Full-text product search      |
| products    | idx_products_hfss           | B-tree (is_hfss, is_featured)         | HFSS filtering for promotions |
| orders      | idx_orders_customer         | B-tree (customer_id, created_at DESC) | Customer order history        |
| orders      | idx_orders_store_status     | B-tree (store_id, status)             | Admin order management        |
| orders      | idx_orders_rider            | B-tree (rider_id, status)             | Rider delivery queue          |
| addresses   | idx_addresses_location      | GiST (location)                       | Delivery zone validation      |
| order_items | idx_order_items_order       | B-tree (order_id)                     | Order detail retrieval        |

Table 5.9: Database Indexes

## 5.4 Row Level Security Policies

Row Level Security (RLS) is enabled on all tables. Policies are defined using PostgreSQL's policy system, which evaluates conditions on every query. The auth.uid() function returns the authenticated user's ID from the JWT token, and custom helper functions retrieve the user's role and store\_id from the profiles table for efficient policy evaluation.

| Table  | Role  | Select         | Insert | Update         | Delete         |
|--------|-------|----------------|--------|----------------|----------------|
| stores | owner | Own store only | Yes    | Own store only | Own store only |



|                     |               |                         |               |                    |                    |
|---------------------|---------------|-------------------------|---------------|--------------------|--------------------|
| stores              | manager       | Own store only          | No            | Own store only     | No                 |
| stores              | customer      | Active stores only      | No            | No                 | No                 |
| profiles            | owner         | Own store staff         | Own profile   | Own store staff    | No                 |
| profiles            | customer      | Own profile only        | Own profile   | Own profile only   | No                 |
| products            | owner/manager | Own store products      | Yes           | Own store products | Own store products |
| products            | customer      | Available products only | No            | No                 | No                 |
| orders              | owner/manager | Own store orders        | No            | Own store orders   | No                 |
| orders              | customer      | Own orders only         | Own orders    | No                 | No                 |
| orders              | picker/rider  | Assigned orders         | No            | Assigned orders    | No                 |
| rider_verifications | owner/manager | Own store riders        | Own documents | Approve/reject     | No                 |
| rider_verifications | rider         | Own documents only      | Own documents | No                 | No                 |

Table 5.10: RLS Policy Summary

## 5.5 Authentication and Session Handling

Supabase Auth manages authentication using JWT (JSON Web Tokens) with automatic refresh. When a user logs in, Supabase issues an access token (1-hour expiry) and a refresh token. The access token is included in every API request and database query, enabling RLS policies to evaluate based on the authenticated user's identity. The refresh token is used to obtain new access tokens without requiring re-authentication, providing a seamless user experience.

On the web, tokens are stored in HTTP-only cookies for CSRF protection. On the React Native mobile app, tokens are stored in the device's secure keychain (iOS Keychain / Android Keystore) via the Supabase React Native Auth client. Session invalidation on password change or admin action is handled by Supabase's built-in session revocation, which blacklists the old JWT tokens and forces re-authentication.

## 5.6 Migration Strategy

Database migrations are managed through Supabase's built-in migration system. Each migration is a timestamped SQL file stored in the project's supabase/migrations/ directory, enabling version-controlled schema evolution. The initial migration creates all core tables, indexes, and RLS policies. Subsequent migrations add features incrementally without breaking existing functionality. Supabase's CLI provides

commands for creating, applying, and reverting migrations, supporting both local development and production deployment workflows.

Seed data scripts are maintained separately in `supabase/seed.sql` for development and testing environments, including sample products, categories, and test user accounts. Production deployments use a controlled migration process: migrations are first applied to a staging Supabase project for validation, then promoted to production after QA verification. Rollback procedures are defined for each migration to ensure rapid recovery from schema issues.

## 6. Implementation Plan

### 6.1 Overview

The implementation plan is structured across 5 phases spanning 8 weeks, aligned with the project deadline of June 15, 2026. The timeline is designed for a team of 2 developers (Kiran and Akshat, both Full Stack Devs) working in parallel where possible. The use of Supabase Auth instead of custom authentication saves approximately 2 weeks, which is reallocated to quality assurance and polish. Each phase has clear deliverables, acceptance criteria, and dependency mappings to ensure smooth handoffs between phases.

### 6.2 Phase 1: Setup and Authentication (Week 1-2)

| Task                                                   | Owner  | Days | Deliverable                               | Acceptance Criteria                         |
|--------------------------------------------------------|--------|------|-------------------------------------------|---------------------------------------------|
| Initialize Next.js project with TypeScript strict mode | Kiran  | 1    | Repo with App Router                      | ESLint, Prettier, TypeScript strict passing |
| Set up Supabase project (dev + staging)                | Akshat | 1    | Supabase project URLs                     | PostGIS extension enabled, RLS on           |
| Configure Tailwind CSS + design tokens                 | Kiran  | 1    | tailwind.config.ts with all design tokens | Colors, spacing, typography match UI brief  |
| Implement Supabase Auth (email/password + OTP)         | Akshat | 3    | Login, Signup, Forgot Password screens    | Auth flow works end-to-end with JWT         |
| Create profiles table + RLS policies                   | Akshat | 1    | Migration file + seed data                | RLS tested for all roles                    |
| Build layout components (Navbar, Sidebar, Footer)      | Kiran  | 2    | Reusable layout components                | Responsive on mobile, tablet, desktop       |

|                                          |        |   |                                   |                                     |
|------------------------------------------|--------|---|-----------------------------------|-------------------------------------|
| Set up Zustand stores (auth, cart, UI)   | Kiran  | 1 | Store files with TypeScript types | Store hydration from Supabase works |
| CI/CD pipeline (GitHub Actions + Vercel) | Akshat | 1 | Auto-deploy on push to main       | Preview deployments for PRs         |

Table 6.1: Phase 1 Tasks

## 6.3 Phase 2: Core Features (Week 3-6)

| Task                                           | Owner  | Days | Deliverable                           | Acceptance Criteria                       |
|------------------------------------------------|--------|------|---------------------------------------|-------------------------------------------|
| Create categories table + admin CRUD           | Akshat | 2    | Category management in admin          | CRUD works, nested categories supported   |
| Create products table + admin CRUD             | Akshat | 3    | Product management in admin           | HFSS flag toggle, VAT category selector   |
| Build product catalog browsing (customer)      | Kiran  | 4    | Home, Category, Product Detail pages  | Pagination, search, filters work          |
| Implement search with PostgreSQL trigram       | Kiran  | 2    | Search API + autocomplete UI          | Search returns results within 200ms       |
| Build shopping cart (add, remove, quantity)    | Kiran  | 3    | Cart page with substitute preferences | Cart persisted across sessions            |
| Integrate Stripe Checkout                      | Akshat | 3    | Checkout flow with payment            | Test payment succeeds/fails gracefully    |
| Create orders table + order placement          | Akshat | 3    | Order creation after payment          | Order items, VAT, delivery fee calculated |
| Build order status tracking (real-time)        | Kiran  | 3    | Order tracking with WebSocket         | Status updates appear within 500ms        |
| Implement delivery pricing engine              | Akshat | 2    | Backend delivery fee calculation      | Threshold model works with admin config   |
| Build admin dashboard (analytics, management)  | Kiran  | 4    | Dashboard with charts and tables      | CRUD for products, orders, riders         |
| Create rider_verifications table + upload flow | Akshat | 2    | Rider document upload + admin review  | Rider blocked until approved              |

Table 6.2: Phase 2 Tasks

## 6.4 Phase 3: Secondary Features (Week 7-8)

| Task                                      | Owner  | Days | Deliverable                       | Acceptance Criteria                    |
|-------------------------------------------|--------|------|-----------------------------------|----------------------------------------|
| React Native Expo project setup           | Kiran  | 2    | Expo project with navigation      | App runs on iOS + Android simulators   |
| Mobile product catalog + cart             | Kiran  | 4    | Browse and cart on mobile         | Same functionality as web              |
| Mobile checkout + payment                 | Kiran  | 2    | Stripe Checkout on mobile         | Payment completes successfully         |
| Picker app (pick list, item confirmation) | Akshat | 3    | Picker mobile screens             | Picker can update item status          |
| Rider app (delivery queue, navigation)    | Akshat | 4    | Rider mobile screens              | Rider can accept and complete delivery |
| Push notifications (Expo + SendGrid)      | Akshat | 2    | Order status push + email         | Notifications received within 5s       |
| Order history + reorder                   | Kiran  | 2    | Order history page with reorder   | Reorder adds all items to cart         |
| Address management (add/edit/delete)      | Kiran  | 2    | Address book with postcode lookup | Postcode validation works              |

Table 6.3: Phase 3 Tasks

## 6.5 Phase 4: Polish and Testing (Week 9-10)

| Task                                      | Owner  | Days | Deliverable                    | Acceptance Criteria               |
|-------------------------------------------|--------|------|--------------------------------|-----------------------------------|
| Unit tests (Jest + React Testing Library) | Both   | 3    | Test suite with > 70% coverage | All critical paths tested         |
| Integration tests (Supabase + Stripe)     | Both   | 2    | API integration test suite     | Payment, order, auth flows tested |
| E2E tests (Playwright for web)            | Kiran  | 2    | E2E test suite                 | Full customer journey tested      |
| Mobile testing (Expo + manual QA)         | Akshat | 2    | Test reports for iOS + Android | No P0/P1 bugs remaining           |
| Performance optimization (Lighthouse)     | Kiran  | 2    | Performance audit report       | Lighthouse score > 90             |
| Accessibility audit (WCAG 2.1 AA)         | Akshat | 2    | Accessibility audit report     | All critical issues resolved      |

|                                         |      |   |                       |                                    |
|-----------------------------------------|------|---|-----------------------|------------------------------------|
| HFSS compliance verification            | Both | 1 | HFSS test report      | Flagged items excluded from promos |
| Security review (RLS, input validation) | Both | 2 | Security audit report | No critical vulnerabilities        |

Table 6.4: Phase 4 Tasks

## 6.6 Phase 5: Deployment and Launch (Week 11)

| Task                                         | Owner  | Days | Deliverable                             | Acceptance Criteria                        |
|----------------------------------------------|--------|------|-----------------------------------------|--------------------------------------------|
| Production Supabase setup                    | Akshat | 1    | Production database + auth              | All migrations applied, RLS active         |
| Vercel production deployment                 | Kiran  | 1    | Live web app URL                        | SSL, CDN, environment variables configured |
| EAS Build + App Store submission             | Kiran  | 2    | iOS + Android builds submitted          | App Store review in progress               |
| Monitoring setup (Vercel Analytics + Sentry) | Akshat | 1    | Error tracking + performance monitoring | Alerts configured for P0 errors            |
| Launch checklist verification                | Both   | 1    | Signed-off launch checklist             | All items verified and approved            |

Table 6.5: Phase 5 Tasks

## 6.7 Risk Assessment

| Risk                          | Likelihood | Impact | Mitigation                                                              |
|-------------------------------|------------|--------|-------------------------------------------------------------------------|
| App Store rejection (iOS)     | Medium     | High   | Follow Apple review guidelines strictly; submit 2 weeks before deadline |
| Stripe 3D Secure friction     | Low        | Medium | Use Stripe's optimized SCA flow; test with UK test cards                |
| Supabase free tier limits     | Medium     | Medium | Monitor usage; upgrade to Pro (\$25/month) if limits approached         |
| PostGIS query performance     | Low        | High   | Add GiST indexes; test with 10,000+ product dataset                     |
| React Native Expo limitations | Low        | Medium | Use Expo dev client for custom native modules if needed                 |

|                                 |        |      |                                                           |
|---------------------------------|--------|------|-----------------------------------------------------------|
| Team capacity (2 devs, 8 weeks) | Medium | High | Strict MoSCoW prioritization; defer "Could Have" features |
|---------------------------------|--------|------|-----------------------------------------------------------|

Table 6.6: Risk Assessment

## 6.8 Testing Strategy

- **Unit Tests:** Jest + React Testing Library for frontend components, Jest for backend utility functions. Target: > 70% code coverage for critical modules (auth, cart, checkout, VAT calculation).
- **Integration Tests:** Supabase local development environment for database integration testing. Stripe test mode for payment flow testing. Target: All API endpoints have integration tests.
- **E2E Tests:** Playwright for web application end-to-end testing covering the complete customer journey (signup, browse, cart, checkout, order tracking). Target: 10+ critical user flows automated.
- **Mobile Testing:** Expo + Detox for React Native automated testing. Manual QA on iOS (iPhone 12, 14, 15) and Android (Pixel 7, Samsung Galaxy S23) devices. Target: Zero P0/P1 bugs at launch.
- **Performance Testing:** k6 load testing for API endpoints. Target: 500 concurrent users with p95 response time < 200ms.
- **Security Testing:** OWASP ZAP for automated vulnerability scanning. Manual review of RLS policies and input validation. Stripe radar for payment fraud detection.

## 6.9 Timeline Summary

| Phase                       | Duration  | Key Deliverables                                        | Team Allocation                                  |
|-----------------------------|-----------|---------------------------------------------------------|--------------------------------------------------|
| Phase 1: Setup + Auth       | Week 1-2  | Project scaffold, Supabase Auth, layout components      | Kiran (frontend) + Akshat (backend)              |
| Phase 2: Core Features      | Week 3-6  | Catalog, cart, checkout, orders, admin dashboard        | Kiran (customer flow) + Akshat (backend + admin) |
| Phase 3: Secondary Features | Week 7-8  | Mobile app, picker app, rider app, notifications        | Kiran (customer mobile) + Akshat (staff mobile)  |
| Phase 4: Polish + Testing   | Week 9-10 | Test suites, performance, accessibility, security       | Both (parallel testing)                          |
| Phase 5: Deploy + Launch    | Week 11   | Production deployment, app store submission, monitoring | Both (coordinated launch)                        |

Table 6.7: Timeline Summary

**[VERIFY: Confirm 8-week timeline with 2 developers is feasible given team availability and other project commitments]**

**[VERIFY: Confirm App Store review timeline (1-2 weeks) is factored into the launch plan]**

# Items Requiring Verification

The following items have been flagged with [VERIFY] markers throughout the document. These represent assumptions or decisions that require confirmation from the project stakeholders before they can be finalized.

1. **[PRD]** Store-specific POS integration requirements - confirm whether any existing POS system needs to be connected in V1
2. **[PRD]** Expected initial SKU count and catalog size - confirm 2,000+ SKUs is accurate for launch
3. **[TRD]** Confirm Vercel free tier bandwidth (100 GB/month) is sufficient for projected launch traffic
4. **[UI/UX]** Confirm whether the brand has an existing logo, color scheme, or style guide that must be followed
5. **[Impl Plan]** Confirm 8-week timeline with 2 developers is feasible given team availability and other project commitments
6. **[Impl Plan]** Confirm App Store review timeline (1-2 weeks) is factored into the launch plan